Day 72: Looking Inside Attention — Alignment Heatmaps on the Toy Task

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 3 | Complete study guide

Watch: Attention in transformers, step-by-step | Deep Learning Chapter 6 (3Blue1Brown) | Attention mechanism: Overview (Google Cloud Tech)

Script: `day72_attention_heatmaps.py` · every number in this guide is copied by program from `day72_run_output.txt`, the real output of that script.

Day 71's attention model writes every digit of a 20-digit sequence correctly. At each output step it also produces a row of weights over the input positions: a record of where it looked. Today reads those records. The largest weight lands on the right input digit for 1.000 of COPY outputs; the rest of the weight goes mainly to the next encoder state, which a probe shows still holds the digit. The alignment forms during training together with accuracy. On sequences longer than any it was trained on, the alignment breaks after about position 20, and so does the output.

0. Where today fits

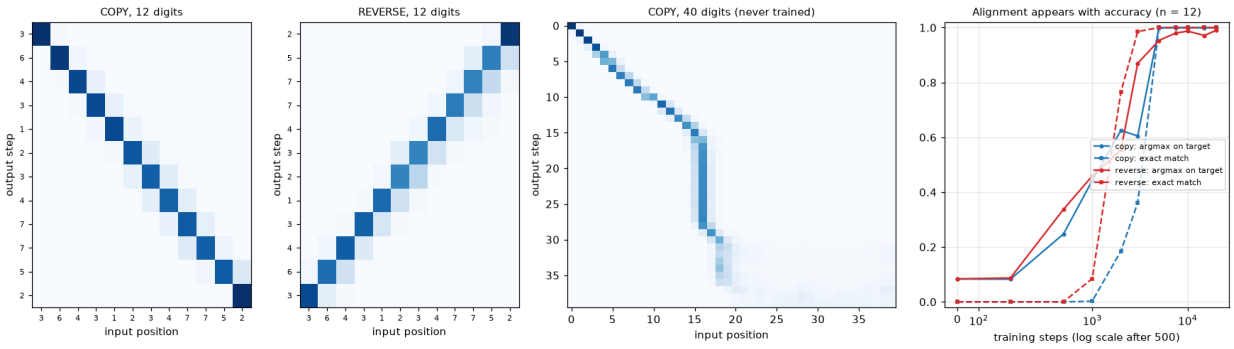
DEFINITION — start here, no prior knowledge assumed

An **alignment** says which input position each output depends on. In attention it is the matrix of weights $\alpha_{j,i}$: row j is output step j, column i is input position i, and each row sums to 1. Drawn as a **heatmap** (dark = large weight), a COPY alignment should be a diagonal and a REVERSE alignment an anti-diagonal.

REAL-WORLD SCENARIO

Eye-tracking a translator: recording where their eyes are on the source text while they write each word of the translation. If they look at the right word at the right time, the recording traces a clean path.

REAL GRAPH



Left to right: a 12-digit COPY (diagonal), a 12-digit REVERSE (anti-diagonal), a 40-digit COPY the model never trained on (the path stalls and dissolves), and alignment during training.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. One alignment, as numbers

DEFINITION — start here, no prior knowledge assumed

The weights of a single 12-digit example, printed as percentages, one row per output step. The TARGET of an output step is the input position holding the digit it should write: the same position for COPY, the mirrored one for REVERSE.

REAL-WORLD SCENARIO

Reading the eye-tracker's raw log for one sentence before looking at averages.

MATHEMATICS

$$\alpha_{j,i} = softmax_i(v_a \cdot \tanh(s_{j-1} W_a + h_i U_a + b_a)), \quad \Sigma_i \alpha_{j,i} = 1 \quad (Day\ 71)$$

$$target:\ COPY\ i^*(j) = j, \quad REVERSE\ i^*(j) = n - 1 - j$$

SPECIFICATION

Models	Day 71's COPY and REVERSE models, hidden 32, retrained with the same seeds and data stream (20,000 steps).
Example	The first of Day 70's fixed 12-digit test sequences.

Why this test

Averages can hide how individual rows look; one example shows the shape directly.

IMPLEMENTATION

```
def alignment_stats(net, task, n, m=300):
    Xs, _, Y, src = fixed_test(n, task, m)
    pred, W = net.generate(Xs, n + 1, return_weights=True)
    W = W[:, :n]
    target = target_position(task, n)
    argmax = W.argmax(axis=2)
    offset = argmax - target[None, :]
    on_target = W[:, np.arange(n), target]
    entropy = -(W * np.log(W + 1e-12)).sum(axis=2)
    correct = pred[:, :n] == Y[:, :n]
    return dict(offset=offset, on_target=on_target, entropy=entropy, correct=correct, W=W,
        src=src, pred=pred)
```

ACTUAL OUTPUT

```
COPY: input [3, 6, 4, 3, 1, 2, 3, 4, 7, 7, 5, 2] -> written [3, 6, 4, 3, 1, 2, 3, 4, 7, 7, 5, 2]
   in:   3   6   4   3   1   2   3   4   7   7   5   2
out 3  99   1   .   .   .   .   .   .   .   .   .   .
out 6   .  96   3   .   .   .   .   .   .   .   .   .
out 4   .   2  90   8   .   .   .   .   .   .   .   .
out 3   .   .   2  89   8   1   .   .   .   .   .   .
out 1   .   .   .   3  90   6   1   .   .   .   .   .
out 2   .   .   .   .   2  85  12   1   .   .   .   .
out 3   .   .   .   .   .   7  82  11   .   .   .   .
out 4   .   .   .   .   .   .   9  81  10   .   .   .
out 7   .   .   .   .   .   .   .   7  84   8   .   .
out 7   .   .   .   .   .   .   .   .  12  82   6   .
out 5   .   .   .   .   .   .   .   .   .   3  83  13
out 2   .   .   .   .   .   .   .   .   .   .   . 100
REVERSE: input [3, 6, 4, 3, 1, 2, 3, 4, 7, 7, 5, 2] -> written [2, 5, 7, 7, 4, 3, 2, 1, 3, 4, 6,
    3]
   in:   3   6   4   3   1   2   3   4   7   7   5   2
out 2   .   .   .   .   .   .   .   .   .   .   3  97
out 5   .   .   .   .   .   .   .   .   .   9  70  21
out 7   .   .   .   .   .   .   .   .   4  68  25   2
out 7   .   .   .   .   .   .   .   4  71  22   3   .
out 4   .   .   .   .   .   .   4  77  14   4   1   .
out 3   .   .   .   .   .  10  68  19   2   1   .   .
out 2   .   .   .   .   3  68  28   1   .   .   .   .
out 1   .   .   .   1  78  20   2   .   .   .   .   .
out 3   .   .   2  81  16   1   .   .   .   .   .   .
out 4   .   4  83  13   1   .   .   .   .   .   .   .
out 6   3  77  20   .   .   .   .   .   .   .   .   .
out 3  90   9   1   .   .   .   .   .   .   .   .   .
Actual output from this lesson's run.
```

What it means

COPY is a clean diagonal: each output looks almost only at its own input digit (99 and 96 on the first rows), with a little weight on the neighbour to the right. REVERSE is the mirror image, but softer: rows such as the second put 70 on the target and 21 on the next input position. Both models wrote the sequence correctly.

WHY CHOOSE THIS?

Shows exactly what the model did on one input, with nothing averaged away.

WHY NOT THIS? (TRADEOFFS)

One example can be unusual; Section 2 measures the whole test set.

2. Alignment over the whole test set

DEFINITION — start here, no prior knowledge assumed

Three measurements per output digit, from Day 72's exercise: **argmax on target** (is the largest weight on the right position?), **weight on target**, and **entropy**, how spread out the row is. Also the weight on the encoder state just **after** the target in reading order (“next”) and just before it (“previous”).

REAL-WORLD SCENARIO

Summarising a thousand eye-tracking recordings: how often the reader looked at the right word, and where else the eyes went.

MATHEMATICS

entropy of a row: $H = -\Sigma_i \alpha_i \log \alpha_i$; *0 if all weight is on one position, log n if spread evenly*

SPECIFICATION

Data	300 fixed test sequences each of 8, 12 and 20 digits; three seeds per task; greedy decoding.

Why this test

It tells whether the diagonal of Section 1 is typical, and where the leftover weight goes.

IMPLEMENTATION

```
nxt, prv = tgt + 1, tgt - 1
ok_n, ok_p = nxt < n, prv >= 0
rows.append([(st["offset"] == 0).mean(), st["on_target"].mean(),
             W[:, j[ok_n], nxt[ok_n]].mean(), W[:, j[ok_p], prv[ok_p]].mean(),
```

ACTUAL OUTPUT

```
task        n  argmax on target  weight: target    next  previous  entropy   exact
copy        8             1.000           0.896   0.071     0.044    0.373   1.000
copy       12             1.000           0.857   0.079     0.071    0.472   1.000
copy       20             1.000           0.806   0.108     0.088    0.605   1.000
reverse     8             0.987           0.771   0.201     0.033    0.633   1.000
reverse    12             0.990           0.734   0.219     0.041    0.728   1.000
reverse    20             0.983           0.681   0.237     0.050    0.874   1.000
Actual output from this lesson's run.
```

What it means

For COPY the largest weight is on the target for every output digit tested (1.000 at all three lengths); for REVERSE for 0.983 to 0.990. The weight on the target falls as sequences get longer (0.896 to 0.806 for COPY) and the rows spread out (entropy 0.373 to 0.605), yet every sequence is still written correctly: the decoder does not need a perfectly sharp look, only the largest weight in the right place.

The leftover weight is not random. For REVERSE it goes mostly to the *next* encoder state (0.201 to 0.237) rather than the previous one (0.033 to 0.050). For COPY it is split more evenly between the two. Section 3 asks which of these states actually hold the digit.

WHY CHOOSE THIS?

Three numbers per row summarise thousands of heatmap rows, comparably across lengths and tasks.

WHY NOT THIS? (TRADEOFFS)

Argmax-on-target ignores how close the runner-up is; entropy alone cannot tell a sharp right look from a sharp wrong one (Day 72's exercise shows a sharp but shifted matrix).

3. Why the next encoder state?

DEFINITION — start here, no prior knowledge assumed

A **probe** asks what a representation contains by training a simple classifier to read it (Day 70). Here: can the digit at input position 8 be read from encoder state h_7 (computed before it), h_8, or later states?

REAL-WORLD SCENARIO

Checking a reader's notes: is a word still mentioned in the notes taken one line, two lines, four lines after it was read?

SPECIFICATION

Probe	Day 70's linear softmax probe on one encoder state of Day 71's COPY encoder (seed 0); 20-digit inputs; 3,000 training and 1,000 test sequences.

Why this test

If the next state also holds the digit, weight placed there is not a mistake: it reads the same information.

IMPLEMENTATION

```
Htr, Hte = net.encode(Xtr)[0], net.encode(Xte)[0]
p = 7
print(f"\n{'probe reads x_8 from':<22}" + "".join(f"{'h_' + str(q + 1):>8}" for q in [p - 1, p, p
    + 1, p + 2, p + 4]))
acc = [(fit_probe(Htr[:, q], src_tr[:, p])(Hte[:, q]) == src_te[:, p]).mean() for q in [p - 1, p,
    p + 1, p + 2, p + 4]]
```

ACTUAL OUTPUT

```
probe reads x_8 from       h_7     h_8     h_9    h_10    h_12
accuracy                 0.131   1.000   0.999   0.601   0.357
guessing: 0.125
Actual output from this lesson's run.
```

What it means

The encoder state at the digit's own position holds it perfectly (1.000), the next state almost as well (0.999), and later states less and less (0.601, then 0.357). The state before cannot know it (0.131, guessing is 0.125). An LSTM that reads left to right carries each digit forward for a step or two, so attention spread onto the next state still reads the right digit, while the previous state adds nothing. That explains REVERSE's leftover weight. It does not explain COPY's: COPY also puts up to 0.088 on the previous state, which this probe says holds nothing about the digit; that weight is probably harmless because the target still dominates, but the reason was not tested here.

WHY CHOOSE THIS?

Turns an observation (weight on the next state) into an explanation that can be checked.

WHY NOT THIS? (TRADEOFFS)

A linear probe measures what is easy to read out, not everything a state contains.

4. When does the alignment appear?

DEFINITION — start here, no prior knowledge assumed

The weights saved at 10 points during training, from untrained to 20,000 steps, each measured on the same 12-digit test sequences: how often the largest weight is on the target, next to exact match.

REAL-WORLD SCENARIO

Watching a student learn to read music: when do their eyes start following the notes, compared with when they start playing the piece correctly?

SPECIFICATION

Snapshots	Steps 0, 250, 500, 1,000, 2,000, 3,000, 5,000, 7,500, 10,000, 15,000, 20,000; three seeds per task.

Why this test

Day 59 showed that end-of-training snapshots can mislead; this follows the alignment while it forms.

IMPLEMENTATION

```
def train_with_snapshots(task, hidden, seed, steps=STEPS, snap_at=SNAPSHOTS):
    net = AttnSeq2Seq(hidden=hidden, seed=seed)
    rng = np.random.RandomState(seed + 100)
    snaps = {0: copy.deepcopy(net.P)}
    for step in range(1, steps + 1):
        n = rng.randint(3, 21)
        Xs, Xd, Y, _ = make_batch(rng, 32, n, task)
        net.train_step(Xs, Xd, Y)
        if step in snap_at:
            snaps[step] = copy.deepcopy(net.P)
    return net, snaps
```

ACTUAL OUTPUT

```
  steps  copy: argmax on target   exact  reverse: argmax on target   exact
      0                  0.083   0.000                      0.084   0.000
    250                  0.083   0.000                      0.087   0.000
    500                  0.247   0.000                      0.337   0.000
   1000                  0.442   0.002                      0.459   0.084
   2000                  0.625   0.186                      0.554   0.766
   3000                  0.605   0.361                      0.870   0.986
   5000                  0.998   1.000                      0.953   1.000
   7500                  1.000   1.000                      0.980   1.000
  10000                  1.000   1.000                      0.987   1.000
  15000                  1.000   1.000                      0.971   1.000
  20000                  1.000   1.000                      0.990   1.000
Actual output from this lesson's run.
```

What it means

Untrained, the largest weight is on the target 0.083 of the time: 1 in 12, chance for 12 positions. The alignment forms early and quickly: COPY reaches 0.442 by 1,000 steps while exact match is still 0.002; by 5,000 steps it is 0.998 and exact match 1.000. For REVERSE the two rise together, and exact match reaches 0.986 at 3,000 steps while the alignment is at 0.870. Learning where to look comes first or together with learning what to write; it is not added afterwards.

WHY CHOOSE THIS?

Shows the mechanism being learned, not just its end state.

WHY NOT THIS? (TRADEOFFS)

Snapshot spacing is coarse between 3,000 and 5,000 steps, where most of the change happens.

5. Where it breaks: longer than any training sequence

DEFINITION — start here, no prior knowledge assumed

The COPY models were trained on 3 to 20 digits. Here they meet 30 and 40 digits. By output position: how many digits are written right, and how often the largest weight is on the target.

REAL-WORLD SCENARIO

A reader trained only on short paragraphs, handed a long one: do they lose their place, or do they keep it and still misread?

SPECIFICATION

Data	300 fixed test sequences of 30 and 40 digits; the three COPY seeds.

Why this test

Day 71 measured that exact match falls on these lengths (0.000 at 30 digits). The weights show whether the model loses its place in the input or keeps its place and writes the wrong digit.

IMPLEMENTATION

```
ok = np.mean([st["correct"].mean(axis=0) for st in sts], axis=0)
on = np.mean([(st["offset"] == 0).mean(axis=0) for st in sts], axis=0)
cols = list(range(0, n, 5)) + [n - 1]
print(f"\nn = {n}")
print(f"{'output position':<22}" + "".join(f"{c + 1:>6}" for c in cols))
print(f"{'digit right':<22}" + "".join(f"{ok[c]:>6.2f}" for c in cols))
print(f"{'argmax on target':<22}" + "".join(f"{on[c]:>6.2f}" for c in cols))
right = np.concatenate([st["on_target"][st["correct"]] for st in sts])
wrong = np.concatenate([st["on_target"][~st["correct"]] for st in sts])
```

ACTUAL OUTPUT

```
n = 30
output position            1     6    11    16    21    26    30
digit right             1.00  1.00  0.96  0.70  0.26  0.15  0.07
argmax on target        1.00  0.92  0.96  0.66  0.05  0.00  0.09
weight on target: digits right 0.675 (16355), digits wrong 0.064 (10645)
n = 40
output position            1     6    11    16    21    26    31    36    40
digit right             1.00  0.98  0.75  0.72  0.21  0.14  0.15  0.15  0.03
argmax on target        1.00  0.85  0.70  0.66  0.04  0.00  0.00  0.00  0.04
weight on target: digits right 0.607 (16250), digits wrong 0.045 (19750)
Actual output from this lesson's run.
```

What it means

The model loses its place. Up to about position 16 the largest weight is mostly on the target and the digits are mostly right. After position 20, the longest length it trained on, the alignment collapses (argmax on target 0.05 at position 21 of 30, then 0.00) and the digits drop towards guessing (0.15 and below; 1 in 8 is 0.125). Digits written right had 0.675 of their weight on the target; digits written wrong only 0.064. The third heatmap shows one case: the path stalls near position 16 for a dozen steps, then dissolves.

The attention never learned to find positions beyond those it saw in training. The scores compare the decoder's state with each encoder state, and nothing in either tells the model where position 25 is if it has never had to look there. Positional information is one of the things the Transformer adds explicitly (Week 12).

WHY CHOOSE THIS?

The weights explain the failure: lost alignment, not a decoder that cannot write.

WHY NOT THIS? (TRADEOFFS)

Two lengths and one task; other failure patterns may appear on other tasks.

Interview preparation — technical questions

Q: How do you visualise attention in an encoder-decoder?

A: Collect the weight vector at every output step and draw the (output steps x input positions) matrix as a heatmap.

Q: How would you measure alignment quality when the right alignment is known?

A: Share of rows whose largest weight is on the right position, the average weight on it, and row entropy.

Q: Why might attention weight sit on a neighbouring position?

A: Neighbouring encoder states can hold the same information: here the next state of a left-to-right LSTM still holds the digit (probe 0.999).

Q: Does a high-entropy attention row mean the model is wrong?

A: Not necessarily: rows got more spread out on longer inputs while every sequence was still copied correctly.

Q: How would you diagnose a seq2seq model failing on long inputs?

A: Look at the alignment by output position: here it collapsed after the longest training length.

Interview preparation — theoretical questions

Q: Is attention an explanation of the model's decision?

A: Partly. Here the weights match where the information is and predict errors well, but weights show where the model read from, not how it used what it read; they can be misleading in other models.

Q: Why did the model fail beyond 20 digits?

A: Its scores depend on content and the decoder's state, not on explicit positions, and it never had to find a position beyond 20 in training.

Q: What is soft versus hard attention?

A: Soft attention takes a weighted average of all positions (differentiable, used here); hard attention picks one.

Q: Why is REVERSE's attention less sharp than COPY's?

A: Not fully explained here; one observation is that its leftover weight goes to the next state, which still holds the digit, so spreading costs little.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> The attention weights of Day 71's model form the alignment a person would draw: a diagonal for COPY, an anti-diagonal for REVERSE. The leftover weight goes where the information still is (the next encoder state). The alignment forms early in training, with or ahead of accuracy. On longer inputs than any in training it collapses past the longest training length, and the output fails with it. The weights are a useful window into the model, not just decoration.

- Heatmap: rows are output steps, columns input positions, each row sums to 1.
- COPY: largest weight on the target for 1.000 of digits; REVERSE about 0.99.
- Leftover weight goes to the next encoder state, which still holds the digit (probe 0.999).
- Untrained: 1 in 12 on target (chance); alignment forms by about 5,000 steps.
- Beyond 20 digits the alignment collapses and digits fall to near guessing.
- Measures: argmax on target, weight on target, entropy.

Suggested watch order, if starting from zero

- 1. Attention mechanism: Overview (Google Cloud Tech) — attention and alignment in five minutes.
- 2. Attention in transformers, step-by-step | Deep Learning Chapter 6 (3Blue1Brown) — attention patterns drawn step by step, as a bridge to Week 12.

Day 73 preview

- **Dot-product (Luong) attention.** Today's score was additive: project, add, squash, score. Day 73 builds the simpler score $s \cdot h_i$ from scratch and compares the two on the same tasks.
- Today's exercise script, `day72_exercise.py`, computes the three alignment measurements on hand-built weight matrices with known answers.